UT3 Bloque 3 Integración de Servlets con DAO y Oracle

Material de estudio y prácticas guiadas

En la UT2 ejecutabas una clase main que llamaba a un DAO. En este bloque el DAO se mantiene, pero la llamada comienza en un navegador. Tomcat recibe la petición, ejecuta LibroServlet y el servlet utiliza las clases de acceso a datos que ya conoces.

Dato importante. La tabla LIBROS de esta práctica no crea el identificador por sí sola. Al dar de alta un libro tendrás que enviar un ID_LIBRO válido y no repetido.

```
Navegador
  -> petición HTTP
  -> Tomcat
  -> LibroServlet
  -> LibroDAO
  -> ConnectionFactory
  -> Oracle
  -> ResultSet
  -> LibroDAO.mapearLibro
  -> objetos Libro, Autor y Categoria
  -> LibroServlet
  -> request.setAttribute(...)
  -> forward(...)
  -> JSP proporcionada
  -> respuesta HTML
  -> navegador
```

Cómo trabajar con el material

1. Trabaja siempre sobre el mismo proyecto Maven.
2. Completa una operación y pruébala antes de continuar.
3. No añadas DAOImpl, mappers, services ni repositorios.
4. No modifiques el diseño visual de las JSP salvo el campo de ID necesario para el alta.
5. Cuando falle, sigue el dato desde HTTP hasta Oracle y de vuelta.

1 Arquitectura del bloque

La aplicación utiliza una arquitectura corta. Cada pieza existe porque resuelve un problema concreto y conoce solo la información que necesita.

Pieza	Para qué sirve	Qué no debe hacer
Navegador	Envía HTTP y representa el HTML recibido	Ejecutar Java o SQL del servidor
Tomcat	Despliega la aplicación y coordina servlets	Decidir las reglas de biblioteca
LibroServlet	Interpreta HTTP y coordina los DAO	Escribir SQL o recorrer ResultSet
DAO concreto	Ejecuta JDBC y devuelve objetos Java	Leer request o elegir una JSP
ConnectionFactory	Crea conexiones Oracle	Mantener una conexión compartida

Pieza	Para qué sirve	Qué no debe hacer
JSP preparada	Convierte atributos en HTML	Consultar directamente Oracle

Estructura que debe verse en el proyecto

```
src/main/java/
  conexion/ConnectionFactory.java
  controller/LibroServlet.java
  dao/LibroDAO.java
  dao/AutorDAO.java
  dao/CategoriaDAO.java
  model/Libro.java
  model/Autor.java
  model/Categoria.java

src/main/webapp/
  assets/
  WEB-INF/web.xml
  WEB-INF/views/
```

No hay interfaces DAO, clases DAOImpl, paquete mapper, capa service ni DTO. Los DAO son clases concretas y cada uno contiene su método privado de mapeo. Esta simplificación permite concentrarse en la integración web.

2 Preparación del proyecto y conexión Oracle

Qué hace Maven

Maven compila el código y construye un archivo WAR. La dependencia ojdbc11 queda dentro de WEB-INF/lib para que la aplicación desplegada pueda usar el driver de Oracle. La API de Servlet se marca como provided porque Tomcat ya la aporta.

Por qué se registra el driver

En este entorno con Tomcat 10.1, DriverManager no siempre descubre automáticamente el driver que está dentro de la aplicación web. La carga explícita evita el mensaje No suitable driver found. El bloque static se ejecuta al cargar ConnectionFactory.

```
static {
    try {
        Class.forName("oracle.jdbc.OracleDriver");
    } catch (ClassNotFoundException e) {
        throw new ExceptionInInitializerError(
                "No se ha encontrado el driver JDBC de Oracle"
        );
    }
}
```

No elimines este bloque. Si falta el driver, la aplicación falla antes de intentar la consulta y el mensaje permite distinguir un problema de dependencia de un error SQL.

Comprobación antes de programar

1. Comprueba que Oracle está arrancado y que el servicio de la URL existe.
2. Revisa USER y PASSWORD de ConnectionFactory.
3. Ejecuta Maven clean package y confirma que se genera el WAR.
4. Despliega el proyecto en Tomcat 10.1.
5. Abre la URL con el context path real y termina en /libros.

3 Tomcat y el controlador único

Tomcat crea LibroServlet. El código de la aplicación no ejecuta new LibroServlet. Cuando llega una petición, HttpServlet.service examina el método HTTP y llama a doGet o doPost.

Objeto	Quién lo crea	Duración	Uso
LibroServlet	Tomcat	Se reutiliza	Coordinar peticiones de libros
request	Tomcat	Una petición	Leer parámetros y guardar atributos
response	Tomcat	Una petición	Estado, cabeceras y cuerpo
DAO	init del servlet	Vida del servlet	Operaciones JDBC sin estado HTTP
Connection	ConnectionFactory	Una operación	Comunicación con Oracle

La ruta del servlet

```
@WebServlet("/libros")
public class LibroServlet extends HttpServlet
```

La anotación asocia la ruta /libros con esta clase. El nombre LibroServlet ayuda a las personas que leen el código, pero no forma parte de la URL. extends HttpServlet permite sobrescribir doGet y doPost.

Inicialización

```
private LibroDAO libroDAO;
private AutorDAO autorDAO;
private CategoriaDAO categoriaDAO;

@Override
public void init() {
    libroDAO = new LibroDAO();
    autorDAO = new AutorDAO();
    categoriaDAO = new CategoriaDAO();
}
```

init se ejecuta una vez después de que Tomcat construya el servlet. Se guardan DAO sin estado de petición. No se guarda Connection, request, response ni el libro actual, porque una misma instancia puede atender varias peticiones de forma concurrente.

4 Mini práctica 1 Listar libros

Objetivo. Obtener los libros mediante LibroDAO, guardarlos en el request y entregar la petición a la JSP de listado.

Flujo que debe ocurrir

```
GET /libros
-> Tomcat llama a doGet
-> accion ausente se convierte en listar
-> listar llama a libroDAO.findAll()
-> Oracle devuelve filas
-> LibroDAO devuelve List<Libro>
-> request guarda el atributo libros
-> forward entrega request y response a libros.jsp
-> la JSP produce HTML
```

Trabajo

```
private void listar(HttpServletRequest request,
                    HttpServletResponse response) throws ... {
    // TODO C1
    // 1. Pide la lista al DAO.
    // 2. Publícala con la clave que espera la vista.
    // 3. Haz forward a la JSP de listado.
}
```

No copies un ResultSet en el request. La vista debe recibir objetos del modelo que ya no dependen de una conexión abierta.

Pruebas

1. Abre /libros sin parámetro accion.
2. Comprueba que doGet selecciona listar.
3. Coloca un punto de interrupción antes de setAttribute e inspecciona la lista.
4. Continúa y verifica que la URL permanece en /libros durante el forward.

5 Mini práctica 2 Detalle y parámetros

Objetivo. Interpretar /libros?accion=detalle&id=3, buscar el libro 3 y mostrar una ficha.

Fragmento	Significado
/libros	Ruta asociada al servlet
?	Comienza la query string
accion=detalle	Primer parámetro
&	Separa parámetros
id=3	Segundo parámetro, recibido como String

Trabajo

```
private void detalle(HttpServletRequest request,
                     HttpServletResponse response) throws ... {
    // TODO C2
    // Lee y convierte id.
    // Consulta findById.
    // Responde 404 si el DAO devuelve null.
    // Publica el libro y haz forward a la vista de detalle.
}
```

Casos de prueba

Petición	Resultado esperado
detalle&id=1	200 y ficha si existe
detalle&id=99999	404 porque no hay fila
detalle&id=abc	400 porque no es un entero
accion=desconocida	400 porque no existe esa operación

6 Mini práctica 3 Preparar formularios

Objetivo. Mostrar un formulario de alta o edición con los catálogos de autores y categorías que necesita la vista.

```
GET /libros?accion=nuevo
-> AutorDAO.findAll()
-> CategoriaDAO.findAll()
-> request[autores], request[categorias], request[modo]
-> forward a libro-formulario.jsp
```

TODO C3 C4 y C8

- Crea los atributos autores y categorias con las colecciones devueltas por sus DAO.
- Para alta, publica modo con el valor crear y abre el formulario.
- Para edición, recupera el libro por id, controla el 404, publica libro y usa modo actualizar.
- No escribas SQL en el servlet para rellenar los select.

Parámetro y atributo no son lo mismo

Operación	Origen	Tipo habitual	Ejemplo
getParameter	Navegador	String	idAutor enviado por form
setAttribute	Código servidor	Cualquier objeto	List<Autor> para la JSP
getAttribute	Código servidor	Object	La JSP accede a autores

7 Mini práctica 4 Alta con identificador manual

Objetivo. Insertar un libro respetando el contrato real de la tabla LIBROS: ID_LIBRO es NOT NULL y no tiene identity, secuencia ni trigger que lo complete.

Por qué aparece ORA 01400

```
INSERT INTO LIBROS
    (TITULO, PRECIO, ANIO_PUBLICACION,
     DISPONIBLE, ID_AUTOR, ID_CATEGORIA)
VALUES (?, ?, ?, ?, ?, ?)
```

Esta sentencia no proporciona ID_LIBRO. Si Oracle no tiene un mecanismo que genere el valor, intenta insertar NULL y aplica la restricción NOT NULL. El resultado es ORA 01400. Este error demuestra que el driver y la conexión sí funcionan: la sentencia llegó a Oracle.

TODO de integración

1. Añade al formulario de alta un control numérico obligatorio para ID_LIBRO.
2. Usa como name el parámetro que lee leerId.
3. Haz que leerLibroDelFormulario copie ese valor en idLibro.
4. Incluye ID_LIBRO en la lista de columnas del INSERT.
5. Añade un séptimo marcador y enlaza los valores en el mismo orden que las columnas.

No se muestra la sentencia final porque forma parte de la práctica. Antes de ejecutar, escribe en una tabla los siete números de parámetro y la propiedad asociada a cada uno.

Pruebas obligatorias

Caso	Resultado
ID nuevo y datos válidos	Una fila insertada y redirect al listado
ID repetido	Oracle rechaza la clave primaria
ID vacío	El navegador o el servlet impide el alta
Autor inexistente	Oracle rechaza la clave foránea
Categoría inexistente	Oracle rechaza la clave foránea

8 Mini práctica 5 POST actualización y eliminación

Las operaciones que modifican datos se reciben en doPost. El parámetro accion selecciona crear, actualizar o eliminar. Después de una escritura correcta, el servlet redirige al listado.

Método	Uso en el bloque	Efecto al recargar
GET	Listar, detalle, mostrar formularios	Repite una consulta
POST	Crear, actualizar, eliminar	Podría repetir la escritura
Redirect tras POST	Ordena al navegador hacer GET	Recarga el GET final

TODO C5 C6 y C7

- Crear: construye el Libro completo, llama a insert y redirige.
- Actualizar: construye el Libro con id, llama a update y controla cero filas.
- Eliminar: lee id, llama a delete y controla cero filas.
- No llames a doGet desde doPost. La redirección debe provocar un GET real del navegador.

Flujo de un alta correcta

```
formulario -> POST /libros accion=crear
-> doPost -> crear
-> parámetros String -> objeto Libro
-> LibroDAO.insert -> Oracle
-> response.sendRedirect(contextPath + "/libros")
-> navegador recibe 302
-> navegador envía GET /libros
-> listado actualizado
```

9 Diagnóstico de errores

Síntoma	Causa probable	Comprobación
404 antes de doGet	Context path o ruta incorrectos	URL, despliegue y @WebServlet
No suitable driver	Driver no registrado o no desplegado	WEB-INF/lib y Class.forName
ORA 01017	Usuario o contraseña incorrectos	Constantes de ConnectionFactory
ORA 12514	Servicio Oracle incorrecto	Parte final de la URL JDBC
ORA 01400 ID_LIBRO	INSERT omite el id	Formulario, modelo, SQL y parámetros
ORA 00001	ID repetido	Valor enviado y clave primaria
ORA 02291	Autor o categoría inexistentes	Valores de las claves foráneas
JSP sin datos	Clave de atributo distinta	setAttribute antes de forward
400 numérico	Texto no convertible	Form Data y getParameter
500 genérico	SQLException capturada	Log completo de Tomcat

Método de localización

1. Comprueba si la petición entra en doGet o doPost.
2. Inspecciona los parámetros antes de convertirlos.
3. Entra en el método DAO y lee la sentencia SQL.
4. Relaciona cada interrogación con su llamada ps.set correspondiente.
5. Comprueba el objeto construido por mapearLibro.
6. Antes del forward, revisa los atributos del request.
7. Usa el log de Tomcat para los detalles de SQLException.

10 Práctica final Biblioteca web

Objetivo. Completar y comprobar el CRUD de libros sin añadir capas nuevas.

Orden de trabajo

1. Verifica ConnectionFactory y la carga explícita del driver.
2. Completa findAll, findById y mapearLibro en LibroDAO.
3. Completa listar y detalle en LibroServlet.
4. Carga autores y categorías en los formularios GET.
5. Integra el identificador manual en formulario, modelo e INSERT.
6. Completa update y delete.
7. Comprueba que las escrituras terminan con redirect.
8. Ejecuta la matriz de pruebas y registra cada resultado.

Matriz de aceptación

Operación	Petición	Evidencia
Listado	GET /libros	Tarjetas con autor y categoría
Detalle	GET detalle e id	Ficha del libro
Alta	GET nuevo y POST crear	Nueva fila con ID elegido
Edición	GET editar y POST actualizar	Misma fila con cambios
Eliminación	POST eliminar	Fila eliminada y listado
Errores	IDs y acciones no válidos	400, 404 o 500 coherentes

Actividad de autonomía

Añade una validación para impedir precios negativos antes de llamar al DAO. Si el precio no es válido, devuelve 400 con un mensaje claro. Después explica por qué esta validación no sustituye las restricciones de la base de datos.

11 Mapa final del bloque

```
Navegador
  -> petición HTTP
  -> Tomcat
  -> LibroServlet
  -> LibroDAO
  -> ConnectionFactory
  -> Oracle
  -> ResultSet
  -> LibroDAO.mapearLibro
  -> objetos Libro, Autor y Categoria
  -> LibroServlet
  -> request.setAttribute(...)
  -> forward(...)
  -> JSP proporcionada
  -> respuesta HTML
  -> navegador
```

Ideas que deben quedar claras

- Tomcat crea el servlet, request y response; la aplicación no los instancia.
- La URL y el método HTTP determinan qué código del servlet se ejecuta.
- Los parámetros llegan como texto y deben validarse y convertirse.
- El servlet coordina; el DAO contiene JDBC; la JSP representa datos preparados.
- setAttribute y forward conservan la misma petición.
- sendRedirect hace que el navegador envíe una petición nueva.
- PreparedStatement separa SQL y valores.
- try with resources cierra Connection, PreparedStatement y ResultSet.
- ID_LIBRO debe enviarse porque el esquema utilizado no lo genera.
- Class.forName registra explícitamente el driver Oracle en el entorno Tomcat del curso.